



# WEEK 5 - ASSIGNMENT

Spring 2026

Pol Monné Parera (L30170743)  
pmonneparera@lewisu.edu

# 1. Technical Description

## 1.1 Dataset

The dataset used has been the CIFAR-10, a widely used benchmark dataset with 60,000 32x32 pixels RGB images. Data is organized into ten mutually exclusive classes, which for the purpose of the assignment have been merged into two super-categories for binary classification.

- Animals: bird, cat, horse, deer, dog, frog (label = 0)
- Vehicles: airplane, automobile, ship, truck (label = 1)

Data was organized in 50,000 training samples and 10,000 test samples, with a slight class imbalance (60% animals, 40% vehicles).

## 1.2 Deep Neural Network (DNN)

A DNN is a fully connected feedforward neural network where every neuron in one layer is connected to every neuron in the next. The input images are flattened into 3,072-dimensional vectors before being inputted into the DNN, with ReLU activations introducing nonlinearity after each dense layer. The final neuron is configured to use a sigmoid activation function to output a probability  $[0, 1]$  suitable for binary classification.

## 1.3 Convolutional Neural Network (CNN)

A CNN preserves the spatial structure of images by applying learned filters via Conv2D layers. Each filter slides over the image producing a dot product with local patches, making possible the detection of features such as edges, textures and shapes at different levels of abstraction. Batch normalization is applied after each block in an effort to accelerate convergence and improve generalization.

# 2. Design of the Algorithms

## 2.1 Preprocessing

1. Load CIFAR-10 dataset (50k for training, 10k for testing)
2. Map the original ten class labels to binary classification
3. Normalize pixel values from  $[0, 255]$  to  $[0, 1]$
4. For the DNN: flatten each 32x32x3 image to a 3,072-dimensional vector
5. For the CNN: keep images in their 32x32x3 tensor form

## 2.2 Pseudocode

LOAD CIFAR-10 → binary labels → normalize

DNN: flatten →  $[512 \rightarrow 256 \rightarrow 128 \text{ relu} + \text{dropout}] \rightarrow \text{sigmoid}$

CNN: conv blocks (32,64) + pool/dropout → dense 256 → sigmoid

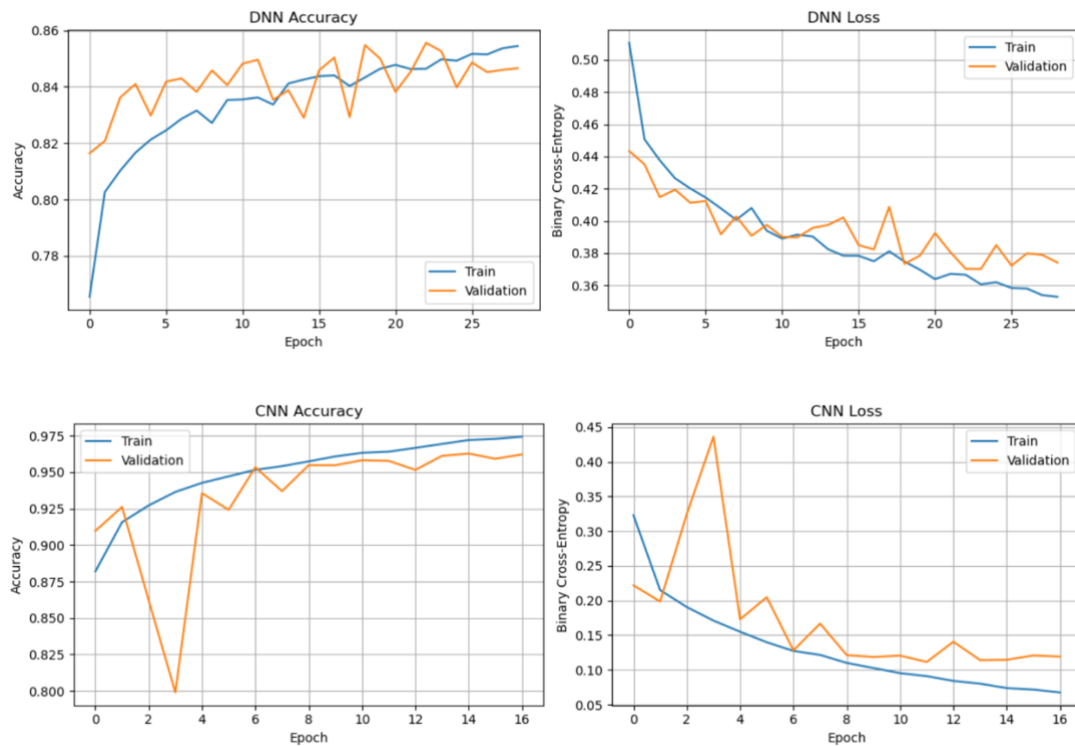
COMPILE (adam, binary\_crossentropy)

TRAIN (DNN: batch128, CNN: batch64, epochs50, early stop)

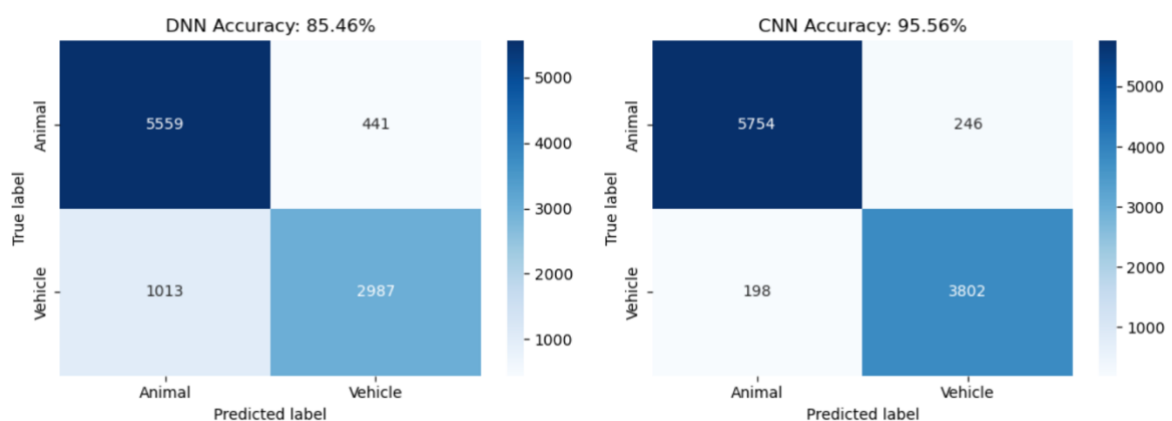
PREDICT (threshold 0.5) → evaluate (accuracy, confusion matrix)

## 3. Results of the Algorithms

### 3.1 Training Curves



### 3.2 Confusion Matrices



### 3.3 Classification Accuracy

| Model | Input          | Test Accuracy |
|-------|----------------|---------------|
| DNN   | Flat 3,072-dim | 85.46%        |
| CNN   | 32x32x3 image  | 95.56%        |

## 4. Analysis of the Results

### 4.1 Did I obtain what I expected

The results were consistent with the previous expectations. Since CNNs are designed to preserve the spatial structure of images through hierarchical filter learning, it was anticipated that they would outperform a flat DNN on a color image dataset. The DNNs 85.46% accuracy is still significant since the binary distinction between animals and vehicles is semantically clear enough for the network to partially learn.

### 4.2 Surprises

The most notable surprise consisted on how the CNN was able to more accurately classify vehicle images, likely because this images share strong geometrical cues (e.g. rectangular shapes or symmetry). These can be easily extracted by convolutional filters.

Furthermore, even though it was anticipated that CNNs would take a longer time to train the difference was surprising. Even more when considering that due to their internal structure the DNN architecture had around 600k more parameters. The CNN took over 11 minutes to train, while the DNN finished in under two minutes.

### 4.3 Conclusions

The CNN architecture is inherently better suited for a CIFAR-10 classification than a flat DNN, as its capabilities to preserve spatial structure of images were decisive as expected. Furthermore, the implementation of early stopping successfully prevented the models of overfitting while obtaining high accuracy results.

Given the work done and possible points of improvement, some potential future work on this project could consist of:

- Add data augmentation to further reduce overfitting and increase accuracy
- Extend to the full 10-class problem and compare multi-class accuracy
- Work with higher resolution images and compare results

## 5. Program Listings

The complete and documented code is provided in the companion python code, named Assignment5\_CNN\_DNN. This code was developed and tested in a Jupyter Notebook environment, but since the assignment required a .py file the notebook was downloaded in that format from Google Colab environment. Below you may find the most relevant code snippets:

```
# Load CIFAR-10
(x_train_raw, y_train_raw), (x_test_raw, y_test_raw) = keras.datasets.cifar10.load_data()

print(f"Train shape: {x_train_raw.shape}, Test shape: {x_test_raw.shape}")
print(f"CIFAR-10 classes: airplane(0), automobile(1), bird(2), cat(3), deer(4),")
print(f"dog(5), frog(6), horse(7), ship(8), truck(9)")

CLASS_NAMES = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

# Binary mapping: vehicles (0,1,8,9) -> 1, animals (2,3,4,5,6,7) -> 0
VEHICLE_CLASSES = {0, 1, 8, 9}

def make_binary(y):
    return (np.isin(y.flatten(), list(VEHICLE_CLASSES))).astype(np.float32)

y_train_bin = make_binary(y_train_raw)
y_test_bin = make_binary(y_test_raw)

print(f"\nBinary label distribution (train): animals={np.sum(y_train_bin==0)}, vehicles={np.sum(y_train_bin==1)}")
```

Figure 1 Preprocessing

```
def build_dnn(input_dim):
    model = Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(512, activation='relu'),
        layers.Dropout(0.4),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.4),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(1, activation='sigmoid')
    ], name='DNN')
    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

    return model

dnn = build_dnn(input_dim=3072)
dnn.summary()
```

Figure 2 DNN architecture

```
def build_cnn(input_shape):
    model = Sequential([
        layers.Input(shape=input_shape),

        # Block 1
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Dropout(0.25),

        # Block 2
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Dropout(0.25),

        # Classifier head
        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ], name='CNN')
    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

    return model

cnn = build_cnn(input_shape=(32, 32, 3))
cnn.summary()
```

*Figure 3 CNN architecture*

## Resources

TensorFlow documentation: [tf.keras.datasets.cifar10](https://www.tensorflow.org/api_docs/python/tf/keras/datasets/cifar10).

[https://www.tensorflow.org/api\\_docs/python/tf/keras/datasets/cifar10](https://www.tensorflow.org/api_docs/python/tf/keras/datasets/cifar10)